



Première année Science du Numérique : Programmation Impérative

Rapport de Projet : Stockage et exploitation de tables de routage

Auteurs :

M^{me} Jeanne MONTMERLE
M. Lucas MOUSSY
M. Alexandre PONCIN

Encadrant :

M. Zouheir HAMROUNI

Version du
12 janvier 2026

Table des matières

1	Résumé	2
2	Introduction	2
3	Découpage modulaire	3
3.1	Relation entre les différents modules	3
3.2	Rôle de chaque module	6
3.2.1	Module Générique : LC	6
3.2.2	Module : Lecture_Fichier	6
3.2.3	Module : Table_Routage	6
3.2.4	Module : Paquet (normal, cache_arbre, cache_lc)	6
3.2.5	Module : Cache_LC	6
3.2.6	Module : Cache_Arbre	6
3.3	Choix particuliers	7
4	Etat de fonctionnement	8
4.1	Routeur Simple	8
4.2	Routeur LL	8
4.3	Routeur LA	8
5	Répartition du travail	9
6	Conclusion	11
6.1	Bilan technique	11
7	Annexe : Bilan personnel	12
7.1	Lucas	12
7.2	Alexandre	12
7.3	Jeanne	12

1 Résumé

Ce rapport de projet présente le fruit du travail fourni dans le cadre de la création d'une table de routage d'un routeur en Ada. Au fil des semaines nous avons pu implanter, évaluer et comparer plusieurs manières d'exploiter une table de routage par le biais de différents raffinages, de modules, scénarios et codes de tests. Nous avons ensuite pu implémenter deux systèmes de cache différents pour notre table, les faire fonctionner et comparer leur efficacité.

2 Introduction

Dans ce projet, nous avons cherché à répondre au mieux à la problématique principale : **optimiser l'acheminement de paquets par un routeur**.

Un routeur est chargé d'orienter le flux des données entre les différentes interfaces jusqu'à les amener vers leur destination finale. Il s'agit d'un rôle fondamental dans l'architecture des réseaux informatique, c'est pourquoi il est vital de concevoir un routeur qui soit le plus efficace possible, afin d'avoir un réseau qui fonctionne fluidement et avec rapidité.

Ici, les défis principaux sont centrés autour de la recherche de la meilleure route dans une table de routage ainsi que l'amélioration de l'efficacité du routeur. Cette efficacité est apportée par l'implémentation d'un **cache**, qui lui-même apporte de nouvelles problématiques de politique de gestion, de cohérence de l'optimisation de base de donnée.

L'enjeu est donc d'élaborer un système capable de router des paquets de manière fiable et performante, en combinant une bonne gestion du cache avec une structure de données arborescente, tout en respectant les règles internes de chaque module.

Dans ce rapport, nous présenterons en premier lieu la relation entre les différents modules avant de décrire en détail le rôle de chacun d'entre eux ainsi que des données utilisées, puis de présenter les choix particuliers qui ont pu être faits.

Nous présenterons ensuite l'état de fonctionnement des différents routeurs.

Nous expliquerons en troisième temps la répartition du travail, puis en quatrième temps nous apporterons chacun un bilan personnel de ce projet, avant de conclure.

3 Découpage modulaire

3.1 Relation entre les différents modules

Les différents modules implémentés permettent de recréer un système de routage réseau, et peuvent être classés dans quatres catégories :

1. Modules générique : **LC**

Ce module fournit une structure de liste chaînée générique pour stocker des éléments, qui sont utilisés dans les modules `Table_Routage`, `Cache_LC`.

2. Modules de stockage : **Table_Routage**, **Cache_LC** et **Cache_Arbre**

Ces trois modules définissent les objets manipulés par le système, respectivement une base de donnée complète de routes et une mémoire rapide de recherche. Ils font appel à LC et sont instanciés dans le module `Paquet`.

3. Modules de service : **Lecture_Fichier**

Ce module permet simplement de traduire un fichier texte en base de donnée. Il est utilisé par `Paquet` et `Table_Routage`.

4. Modules de gestion : **Paquet**

Ce dernier module a pour rôle de coordonner tous les précédents modules ; c'est une sorte de cerveau opérationnel. Il fait appel à tous les autres modules.

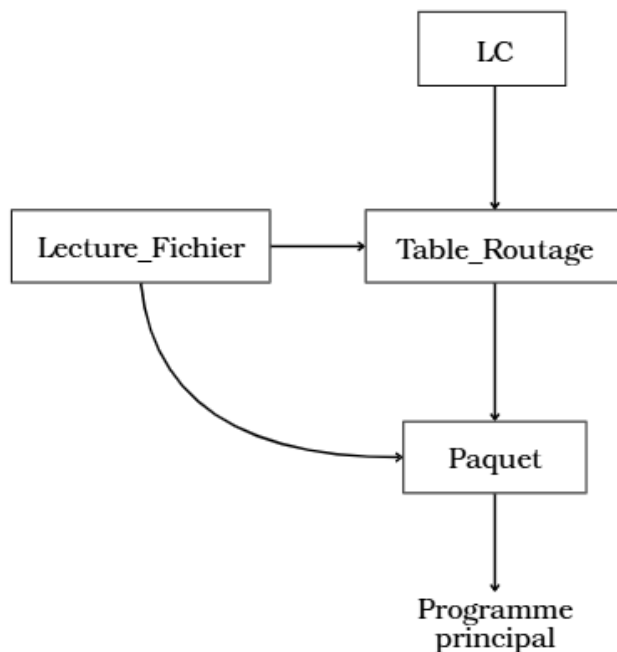


FIGURE 1 – liens d'instanciation entre les différents modules sans caches

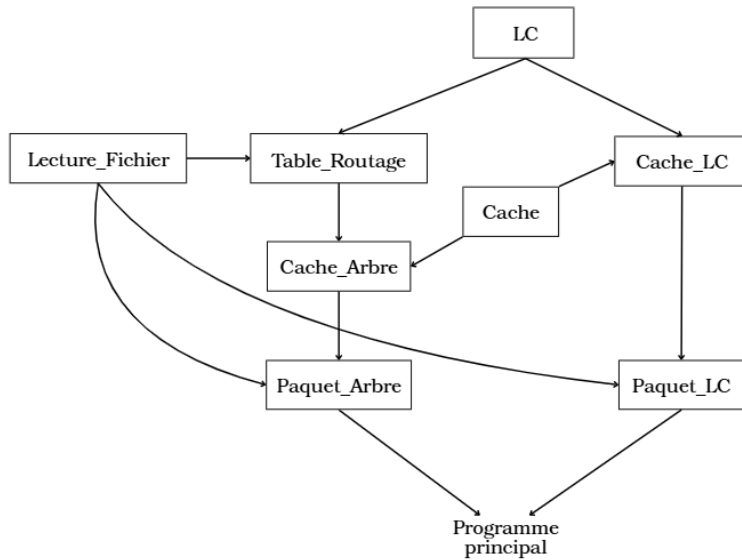


FIGURE 2 – liens d’instanciation entre les différents modules avec les caches

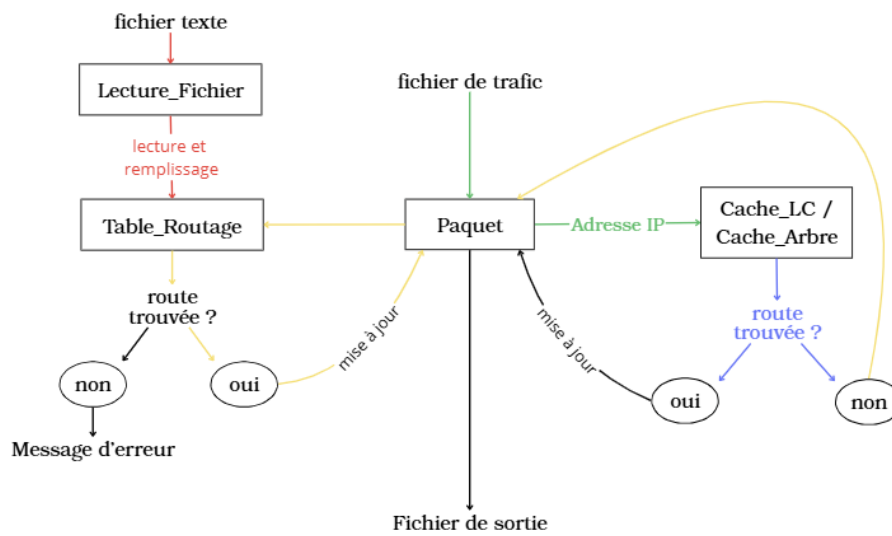


FIGURE 3 – flux de données

Phases du flux :

1. rouge : Initialisation
2. vert : Traitement
3. bleu : Décision
4. jaune : Si route non trouvée, recherche dans la table

Types de données circulant dans le flux :

- **Chaînes de caractères** (String / Unbounded_String) : Entre les fichiers et les modules de haut niveau.
- **T_Adresse_IP** (32-bit Integer) : Entre Paquet, Table_Routage et Cache_Arbre.
- **Discriminant** (Integer) : Flux de contrôle interne au cache pour la gestion du temps/fréquence.
- **Bits** (booléen) : Flux interne au module Cache_Arbre pour la navigation dans les fils.

3.2 Rôle de chaque module

3.2.1 Module Générique : LC

Le module LC est un module de **stockage de données génériques**. Il s'agit du module de plus bas niveau de notre système : en effet, il fournit une structure de données flexible sous la forme de **listes chaînées**. C'est un peu une sorte de "classeur" vide qui permet de ranger des fiches, peu importe ce qu'il y a écrit dessus. Ici, il sert de fondation au module Table_Routage pour stocker la liste des routes.

LC gère les opérations de base comme *Initialiser*, *Enregistrer*, *Supprimer* ou *Taille*, et permet le parcours des données via la procédure interne *Faire_Pour_Chaque*.

3.2.2 Module : Lecture_Fichier

Il s'agit de l'interface de **traduction** de notre système. Son rôle est de traduire le contenu en texte des fichiers en structures de données binaires exploitables par la machine, et permet de remplir la Table_Routage initiale.

3.2.3 Module : Table_Routage

Ce module agit comme la **base de données de référence** du routeur. Il définit ce qu'est une "Route" (Destination, Masque, Interface), agit comme un annuaire complet de tous les réseaux connus par le routeur et a comme rôle de déterminer quelle interface doit être utilisée pour un paquet donné. Cela est permis par **l'algorithme de recherche du plus long préfixe** (*Obtenir_Pire_Masque* dans notre code) qui, lorsqu'on cherche une interface, parcourt toutes les routes pour trouver celle dont le masque est le plus précis (le plus grand) et qui correspond à l'IP.

3.2.4 Module : Paquet (normal, cache_arbre, cache_lc)

Le module Paquet est le **contrôleur principal** ("chef d'orchestre") du fonctionnement du routage. Il fait le lien entre les fichiers de données, le cache et la table de routage. Il possède toute un enchainement de responsabilités :

- Lire le fichier de trafic (paquets.txt).
- Prendre la décision pour chaque ligne :
 - Si c'est une commande, il demande aux modules concernés de s'afficher.
 - Si c'est une IP, il orchestre la recherche : d'abord dans le Cache_LC, et s'il ne trouve pas, dans la Table_Routage.
- Mettre à jour le cache après chaque succès de la table.
- Gérer la cohérence des données et de **l'algorithme de remplacement de cache** selon la politique choisie (FIFO, LRU, LFU).
- Écrire les résultats dans le fichier de sortie.

3.2.5 Module : Cache_LC

Cache_LC (Cache par Liste Chaînée) est comme une **mémoire à court terme** pour le routeur. Il stocke les résultats des recherches récentes pour éviter de recalculer la route la plus précise dans la table de routage, car c'est une opération coûteuse. Ce module a le rôle de rendre la recherche de route plus performante en, contrairement à la table qui cherche par "préfixe", cherchant une correspondance exacte de l'adresse IP, ce qui est beaucoup plus rapide.

3.2.6 Module : Cache_Arbre

Ce dernier module présente une **alternative haute performance au module Cache_LC**. Grâce à l'utilisation d'un **algorithme d'arbre préfixe**, il peut assurer la fonction de cache mais en

utilisant une structure de données beaucoup plus efficace pour le routage.

3.3 Choix particuliers

- plusieurs modules Paquet différents

4 Etat de fonctionnement

4.1 Routeur Simple

Le routeur simple répond parfaitement au cahier des charges.

4.2 Routeur LL

Le routeur LL répond parfaitement au cahier des charges.

4.3 Routeur LA

Le routeur LA fonctionne correctement.

5 Répartition du travail

	nom court	NOM	Prénom			
équipier1	LM	Moussy	Lucas			
équipier2	JM	Montmerle	Jeanne			
équipier3	AP	Poncin	Alexandre			

Quoi ?		Qui ?			
Modules	Sous-programmes	Spécif.	Raffinage	Codage	Test
Routeur (main) (présence de résultat)	Initialiser	AP	AP	AP	AP
	Renvoyer_Utilisation	AP	AP	AP	AP
	Recuperer_Argument	AP	AP	AP	AP
LC générique	Initialiser	LM	LM	LM	JM
	Est_Vide	LM	LM	LM	JM
	Detruire	LM	LM	LM	JM
	Faire_Pour_Chaque	LM	LM	LM	JM
	Enregistrer	LM	LM	LM	JM
	La_Premiere_Valeur	AP		AP	JM
	Donnee_Presente	AP		AP	JM
	Supprimer_Iteratif	AP		AP	JM
	Taille	LM		LM	JM
	Traiter	LM		LM	JM
Table_Routage	Initialiser	LM	LM	LM	JM
	Est_Vide	LM	LM	LM	JM
	Enregistrer	LM	LM	LM	JM
	Detruire	LM	LM	LM	JM
	Afficher_IP	LM	LM	LM	JM
	Afficher_Table	LM	LM	LM	JM
	Trouver_Interface_Sans			AP	JM
	Afficher_Une_Route	LM	LM	LM	JM
lecture_fichier	Recuperer_Destination	LM	LM	LM	JM
	Recuperer_Masque	LM	LM	LM	JM
	Recuperer_Interface	LM	LM	LM	JM
	Est_Paquet	AP		JM	JM
	Recuperer_Paquet	AP		AP	JM
	Sauter_Espace	LM	LM	LM	JM
Suite page suivante...					

Table 1 – suite de la page précédente

Quoi ?		Qui ?			
Modules	Sous-programmes	Spécif.	Raffinage	Codage	Test
	Convertir_Unbounded	LM	LM	LM	JM
	Lecture_table_to_LC	LM	LM	LM	JM
Paquet & Paquet_arbre & Paquet_lc	Traiter	AP	AP	AP & JM	AP/JM
	Traiter_Ligne	AP	AP	AP	AP/JM
	Traiter_Paquet	AP	AP	AP	AP/JM
	Traiter_Commande	AP	AP	JM & AP	AP/JM
	Ecrire_Fichier_Resultat	AP	AP	JM & AP	AP/JM
Cache_LC	Afficher_Cache	LM		AP	JM
	Initialiser	AP		AP	JM
	Est_vide	AP		AP	JM
	Detruire	AP		AP	JM
	Afficher_IP	LM		AP	JM
	Supprimer	AP		AP	JM
	Afficher_Une_Route	LM		AP	JM
	Taille	AP		AP	JM
	Enregistrer	AP		AP	JM
	Supprimer_route	AP		AP	JM
	Actualiser	AP		AP	JM
	Changer_Discriminant	AP		AP	JM
	Trouver_Route	AP		AP	JM
Cache_arbre	Initialiser	LM	LM	LM	JM
	Detruire	LM	LM	LM	JM
	Est_Vide	LM	LM	LM	JM
	Taille	LM	LM	LM	JM
	Enregistrer	LM	LM	LM	JM
	Supprimer_Route	LM	LM	LM	JM
	Afficher_Cache	LM	LM	LM	JM
	Afficher_Une_Route	LM	LM	LM	JM
	Trouver_Route	LM	LM	LM	JM
	Chercher_route	LM	LM	LM	JM
	Changer_Discriminant	AP		AP	JM
	Actualiser	AP		AP	JM
	Faire pour chaque	LM		LM	JM

6 Conclusion

Ce projet avait pour objectif d'optimiser au mieux l'acheminement de paquets par un routeur à l'aide d'une architecture de système en plusieurs modules. Au bout de multiples semaines de travail, nous avons abouti à une implantation fonctionnelle de tous les module : les modules Paquets traitent en effet correctement les fichiers d'entrée, identifient les interfaces grâce à l'algorithme du plus long préfixe et produit les résultats attendus. L'ajout des deux modules de cache Cache_LC et Cache_Arbre a ensuite été effectué et les tests modulaires vérifiés.

Ce projet nous a permis de consolider les concepts de modules, de généricité, de types privés limités et de gestion de la mémoire. On peut aussi noter qu'il nous a appris l'importance d'une architecture modulaire avec les deux types de caches pour permettre la modification et l'interchangeabilité des composants sans modifier le cœur du module Paquet.

6.1 Bilan technique

Efficacité des politiques de cache : Retour sur l'impact des politiques FIFO, LRU et LFU sur le taux de réussite du cache selon les différents jeux de tests.

Gestion de la mémoire : Mention de la robustesse du module LC et de la désallocation systématique des pointeurs pour éviter les fuites mémoires.

Analyse critique et Limites

Complexité de l'Arbre : Reconnaître que si Cache_Arbre est plus rapide, sa gestion (notamment la suppression récursive d'un discriminant minimal) est plus complexe à implémenter que la liste.

Stabilité : Discussion sur la robustesse du parsing dans Lectur_Fichier face à des données mal formées.

Perspectives d'évolution :

Possibilité de créer un module générique pour le paquet.

7 Annexe : Bilan personnel

7.1 Lucas

Intérêt Malgré un intérêt initial limité pour le sujet, la dimension collaborative de ce projet a été très enrichissante. J’ai particulièrement apprécié l’aspect technique lié aux tables de routage et à leur traduction algorithmique. Ce projet m’a également permis de gagner significativement en aisance avec le langage Ada.

- Temps passé à la conception : 3 heures
- Temps passé à l’implantation : une journée
- Temps passé à la mise au point : 3 heures
- Temps passé sur le rapport : 1 heure 30

Total : 31 heures 30

Enseignements tirés de ce projet : Les principaux enseignements tirés de ce projet sont surtout liés à l’organisation et au partage des tâches. Je me suis rendu compte qu’il était vraiment primordial de se fixer des deadlines afin de véritablement avancer et de pouvoir atteindre les objectifs dans le temps imparti.

7.2 Alexandre

Intérêt : Le routage et plus globalement le réseau est un domaine que je ne connaissais pas et qui m’intriguait.

- Temps passé à la conception : 5h
- Temps passé à l’implantation : 20h
- Temps passé à la mise au point : 5h
- Temps passé sur le rapport : 0h (et 3h sur le manuel)

Total : 33h

Enseignements tirés de ce projet : J’ai appris qu’un projet de cette taille peut être très long, heureusement qu’on était 3. Et je sais maintenant comment fonctionne un routeur (mais peut-être une version simplifiée de la réalité).

7.3 Jeanne

Intérêt : Bien que sur un sujet complexe et utile à la compréhension des outils qui nous entoure (table de routage), je n’ai personnellement pas réussi à trouver un intérêt conséquent pour ce projet. J’ai plus été motivée par l’envie d’apporter une contribution équilibrée au projet que par le projet lui-même, surtout lorsqu’il a été apparent que mon niveau en codage est moins bon que celui de mes camarades. J’ai néanmoins toujours pu avancer grâce à l’aide et la patience de mes coéquipiers et en m’investissant plus sur d’autres aspects du projet.

- Temps passé à la conception : 3h
- Temps passé à l’implantation : 13h
- Temps passé à la mise au point : 4h
- Temps passé sur le rapport : 8h

Total : 28h

Enseignements tirés de ce projet : Organisation en groupe et personnelle pour répondre aux besoins

du groupe, compréhension plus en profondeur du langage Ada, des modules et de leur fonctionnement, apprendre à demander de l'aide quand j'avais du mal sur certains concepts, langage LaTeX et utilisation d'OverLeaf.